

Graph Theory Assignment 3 Report

Group 5

- Maulana Anugra Putra (5025251159)
- I Gusti Agung Candra Nugraha (5025251169)
- Hussein Mohammad Mahsun (5025251170)



Institut Teknologi Sepuluh Nopember

Group Homework

Adapted from University of Helsinki Computer Science Department

You are a software engineer working for a city's postal service.

A mail carrier must start at the post office, visit every street exactly once to be efficient, and return to the post office. Such problem can be modeled into graph, with crossings as *nodes* and roads as *edges*

This is simulated in CSES Mail Delivery problem: <https://cses.fi/problemset/task/1691>

Your task

1. Implement Fleury's Algorithm and submit and pass all test.
2. Implement Hierholzer's Algorithm and submit and pass all test.
3. Implement any other eulerian algorithm, then submit and pass all test.

Submission:

1. GitHub repository containing the implementation
2. A PDF report of 1-3, committed in the repository
 - a. Take one sample test case and explain steps of each algorithm until getting final result
 - b. Analyze which algorithm performs the best, based on complexity



Institut Teknologi Sepuluh Nopember its_campus | its.ac.id

1. Fleury's Algorithm

Sample test case

Input:

6 8
1 2
1 3
2 3
2 4
2 6
3 5
3 6
4 5

How Fleury's algorithm works

Starting from the post office (node 1), repeatedly walk across an unused street to a neighboring crossing, but never cross a street that would disconnect the remaining unvisited streets from where you're standing (a bridge), unless it's your only option. Stop when no streets are left. if you're back at node 1, you've found the route.

Pre-check before it starts: an Eulerian circuit exists only if every crossing has an even number of streets, and all streets belong to one connected component. The code checks this first and prints IMPOSSIBLE if it fails.

Trace on the sample:

1-2, 1-3, 2-3, 2-4, 2-6, 3-5, 3-6, 4-5

All degrees are even (1:2, 2:4, 3:4, 4:2, 5:2, 6:2) and the graph is connected, so a circuit exists.

Step	At node	Candidate edges	Bridge check	Edge taken	New position
1	1	1-2, 1-3	Neither is a bridge (graph stays fully connected either way)	1-2	2
2	2	2-3, 2-4, 2-6	None is a bridge	2-3	3
3	3	3-5, 3-6, 3-1	None is a bridge yet	3-5	5
4	5	5-4	Only option (forced)	5-4	4
5	4	4-2	Only option (forced)	4-2	2
6	2	2-6	Only option (forced)	2-6	6
7	6	6-3	Only option (forced)	6-3	3
8	3	3-1	Only option (forced)	3-1	1

Result: 1 2 3 5 4 2 6 3 1

8 edges used, ends back at node 1, so it's a valid Eulerian circuit.

Bridge check mechanic in the code

For an edge (u, v), it does a DFS/BFS from u counting reachable nodes with all edges available, then repeats with that one edge removed. If the reachable count drops, the edge was a bridge (removing it disconnects part of the remaining graph) so it's saved for last.

2. Hierholzer's Algorithm

Sample test case

Input:

6 8
1 2
1 3
2 3
2 4
2 6
3 5
3 6
4 5

How Hierholzer's algorithm works

Starting from the post office (node 1), follow unused streets until you get stuck and return to node 1, forming a preliminary cycle. If there are still unused streets connected to any crossing along your current tour, start a new sub-tour from that crossing, follow unused streets until you get stuck back at that crossing, and splice this new sub-tour directly into your original route. Repeat this splicing process until no unused streets remain.

Trace on the sample:

1-2, 1-3, 2-3, 2-4, 2-6, 3-5, 3-6, 4-5

All degrees are even (1:2, 2:4, 3:4, 4:2, 5:2, 6:2) and the graph is connected, so a circuit exists.

Step	At node	Candidate edges	Action	Edge taken	Path
1	1	1-2, 1-3	Start main tour from node 1; push node 1 and follow edge 1-2	1-2	[1, 2]
2	2	2-3, 2-4, 2-6	Continue tour; push node 2 and follow edge 2-3	2-3	[1, 2, 3]
3	3	3-5, 3-6, 3-1	Continue tour; push node 3 and follow edge 3-5	3-5	[1, 2, 3, 5]
4	5	5-4	Only option; push node 5 and follow edge 5-4	5-4	[1, 2, 3, 5, 4]
5	4	4-2	Only option; push node 4	4-2	[1, 2, 3, 5, 4,

			and follow edge 4-2		2]
6	2	2-6	Main tour at node 2; start sub-tour via remaining edge 2-6	2-6	[1, 2, 3, 5, 4, 2, 6]
7	6	6-3	Continue sub-tour; push node 6 and follow edge 6-3	6-3	[1, 2, 3, 5, 4, 2, 6, 3]
8	3	3-1	Follow edge 3-1 back to start node 1	3-1	[1, 2, 3, 5, 4, 2, 6, 3, 1]

3. Naive / Backtracking Search Algorithm

Sample test case

Input:

6 8
1 2
1 3
2 3
2 4
2 6
3 5
3 6
4 5

How Naive / Backtracking Search Works

From the starting node, the algorithm explores available unused edges recursively using DFS exploration. At each step, it chooses an unused edge, appends it to the path, and moves to the next node. If it reaches a dead end (a node with no remaining unused edges) before all graph edges have been visited, the algorithm recognizes a failure, backtracks by removing the last edge from the path, unmarking it, and tries the next available alternative branch until a complete Eulerian circuit covering all edges is found.

Step	At Node	Candidate Edges	Action	Edge Taken	Path / State
1	1	1-2, 1-3	Start search from node 1; pick first available edge	1-2	[1, 2]
2	2	2-3, 2-4, 2-6	Choose candidate edge 2-3 to explore	2-3	[1, 2, 3]

3	3	3-1, 3-5, 3-6	Choose candidate edge 3-5	3-5	[1, 2, 3, 5]
4	5	5-4	Only available option from node 5	5-4	[1, 2, 3, 5, 4]
5	4	4-2	Only available option from node 4	4-2	[1, 2, 3, 5, 4, 2]
6	2	2-4, 2-6	Choose remaining edge 2-6	2-6	[1, 2, 3, 5, 4, 2, 6]
7	6	6-3	Only available option from node 6	6-3	[1, 2, 3, 5, 4, 2, 6, 3]
8	3	3-1, 3-6	Choose remaining edge 3-1	3-1	[1, 2, 3, 5, 4, 2, 6, 3, 1]
9	1	(None)	Reached node 1, and all edges exhausted. Check completion: All 8 edges successfully visited! Valid Eulerian Circuit found.	Complete	[1, 2, 3, 5, 4, 2, 6, 3, 1]